

Санкт-Петербургский Национальный Исследовательский Университет
Информационных технологий, механики и оптики

Отчет

Дисциплина: Web-программирование
Лабораторная работа 6.

Выполнил: Теремов М.А.

Группа № K3322

Проверила: Марченко Е.В.

Санкт-Петербург

2024

Оглавление

Введение	3
Ход работы	4
Вывод	11

Введение

Цель работы: овладеть практическими навыками и умениями реализации web-серверов и использования сокетов.

Ход работы

1. Задание 1 – отсылка сообщений

В данном задании необходимо реализовать клиентскую и серверную часть приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента. Была использована библиотека `socket`. Код сервера представлен на рисунке 1, код клиента представлен на рисунке 2.

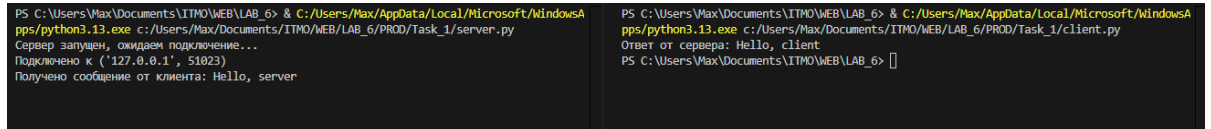
```
PROD > Task_1 > server.py > ...
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 12345
5
6  with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7      s.bind((HOST, PORT))
8      s.listen()
9      print('Сервер запущен, ожидаем подключение...')
10
11     while True:
12         conn, addr = s.accept()
13         with conn:
14             print('Подключено к', addr)
15             data = conn.recv(1024)
16             print('Получено сообщение от клиента:', data.decode())
17             conn.sendall(b'Hello, client')
18
```

Рисунок 1 – Код сервера задание 1

```
PROD > Task_1 > client.py > ...
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 12345
5
6  with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7      s.connect((HOST, PORT))
8      s.send("Hello, server".encode())
9      data = s.recv(1024).decode()
10     print("Ответ от сервера:", data)
11
```

Рисунок 2 – Код клиента задание 1

Результат выполнения кода от лица сервера и клиента представлен на рисунке 3



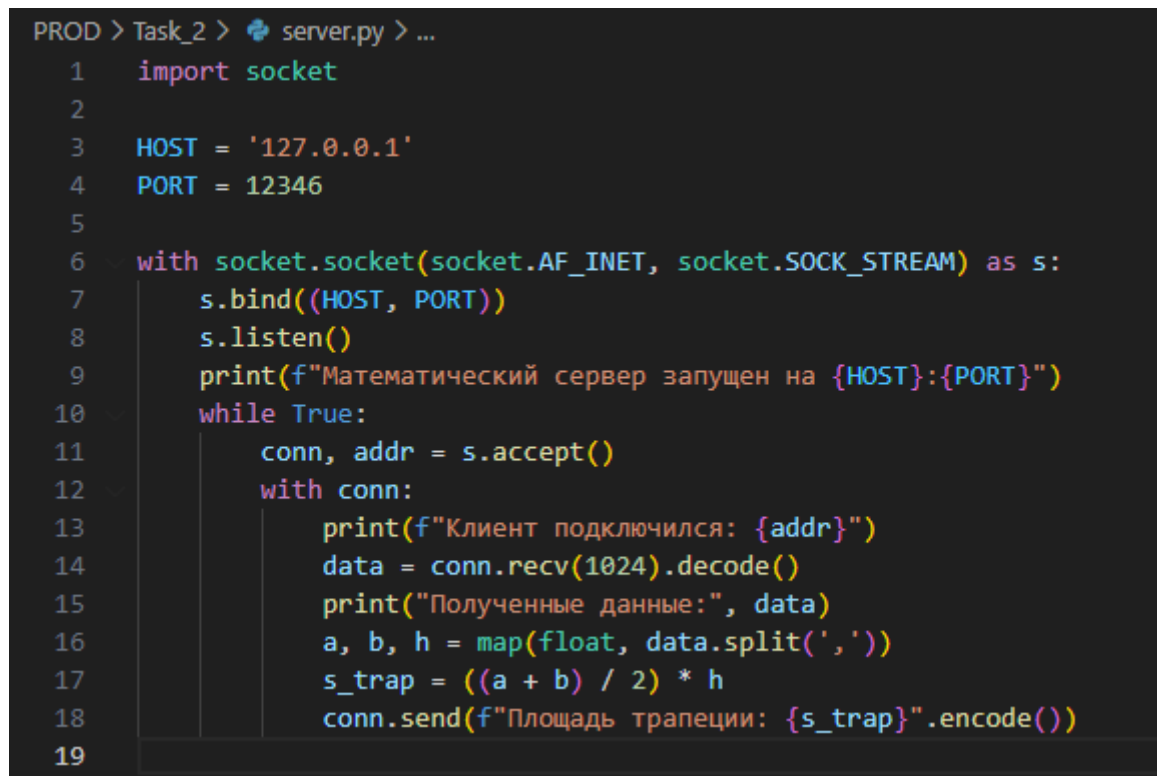
```
PS C:\Users\Max\Documents\ITMO\WEB\LAB_6> & C:\Users\Max\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:\Users\Max\Documents\ITMO\WEB\LAB_6\PROD\Task_1\server.py
Сервер запущен, ожидаем подключение...
Подключено к ('127.0.0.1', 51023)
Получено сообщение от клиента: Hello, server

PS C:\Users\Max\Documents\ITMO\WEB\LAB_6> & C:\Users\Max\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:\Users\Max\Documents\ITMO\WEB\LAB_6\PROD\Task_1\client.py
Ответ от сервера: Hello, client
PS C:\Users\Max\Documents\ITMO\WEB\LAB_6> []
```

Рисунок 3 – Результат выполнения кода со стороны сервера и клиента в задании 1

2. Задание 2 – Поиск площади трапеции.

В данном задании необходимо реализовать клиентскую и серверную часть приложения. Клиент запрашивает у сервера выполнение математической операции, параметры, которые вводятся с клавиатуры. Сервер обрабатывает полученные данные и возвращает результат клиенту. Мой порядковый номер внутри потока WEB 1.1 31, следовательно был выбран вариант с расчетом площади трапеции. По сути выполнение задание очень походе на первое, просто добавлены вычисления. Код сервера представлен на рисунке 4, код клиента представлен на рисунке 5.



```
PROD > Task_2 > server.py > ...
1 import socket
2
3 HOST = '127.0.0.1'
4 PORT = 12346
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7     s.bind((HOST, PORT))
8     s.listen()
9     print(f"Математический сервер запущен на {HOST}:{PORT}")
10    while True:
11        conn, addr = s.accept()
12        with conn:
13            print(f"Клиент подключился: {addr}")
14            data = conn.recv(1024).decode()
15            print("Полученные данные:", data)
16            a, b, h = map(float, data.split(','))
17            s_trap = ((a + b) / 2) * h
18            conn.send(f"Площадь трапеции: {s_trap}".encode())
19
```

Рисунок 4 – Код сервера задание 2

```

PROD > Task_2 > client.py > ...
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 12346
5
6  a = float(input("Введите основание a: "))
7  b = float(input("Введите основание b: "))
8  h = float(input("Введите высоту h: "))
9
10 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
11     s.connect((HOST, PORT))
12     s.send(f"{a},{b},{h}".encode())
13     result = s.recv(1024).decode()
14     print("Ответ от сервера:", result)

```

Рисунок 5 – Код клиента задание 2

Результат выполнения со стороны клиента и сервера представлен на рисунке 6.

<pre> PS C:\Users\Max\Documents\ITMO\WEB\LAB_6> & C:\Users\Max\AppData\Local\Microsoft\WindowsA pps/python3.13.exe c:\Users\Max\Documents\ITMO\WEB\LAB_6\PROD\Task_2\client.py Введите основание a: 12 Введите основание b: 3 Введите высоту h: 4 Ответ от сервера: Площадь трапеции: 30.0 PS C:\Users\Max\Documents\ITMO\WEB\LAB_6> </pre>	<pre> PS C:\Users\Max\Documents\ITMO\WEB\LAB_6> & C:\Users\Max\AppData\Local\Microsoft\WindowsA pps/python3.13.exe c:\Users\Max\Documents\ITMO\WEB\LAB_6\PROD\Task_2\server.py Математический сервер запущен на 127.0.0.1:12346 Клиент подключился: ('127.0.0.1', 51456) Полученные данные: 12.0,3.0,4.0 </pre>
---	--

Рисунок 6 – результат выполнения со стороны клиента и сервера в задании 2

3. Задание 3 – HTML страница

В данном задании необходимо реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html. Код сервера представлен на рисунке 7.

```

PROD > Task_3 > server.py > ...
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 8080
5
6  with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7      s.bind((HOST, PORT))
8      s.listen()
9      print(f"HTTP-сервер запущен на http://{HOST}:{PORT}")
10     while True:
11         conn, addr = s.accept()
12         with conn:
13             request = conn.recv(1024)
14             print(f"Запрос от {addr}:\n{request.decode()}")
15
16             with open("PROD\\Task_3\\index.html", "r", encoding="utf-8") as file:
17                 html_content = file.read()
18
19             response = (
20                 "HTTP/1.1 200 OK\n"
21                 "Content-Type: text/html\n"
22                 f"Content-Length: {len(html_content.encode())}\n\n"
23                 f"{html_content}"
24             )
25             conn.sendall(response.encode())
26

```

Рисунок 7 – Код сервера в задании 3

При переходе по адресу `http://127.0.0.1:8080` высвечивается следующая страница (Рисунок 8).



Привет от самодельного HTTP-сервера!

Рисунок 8 – HTML страница из задания 3

4. Задание 4 – многопользовательский чат

В данном задании необходимо реализовать многопользовательский чат. Для этого использовалась библиотека websockets в python. Код серверной части представлен на рисунке 9.

```
PROD > Task_4 > server.py > ...
1  import asyncio
2  import websockets
3
4  clients = {} # websocket: nickname
5
6  async def handler(websocket):
7      try:
8          await websocket.send("Введите никнейм:")
9          nickname = await websocket.recv()
10
11         # Проверка на уникальность
12         if nickname in clients.values():
13             await websocket.send("Никнейм уже занят. Перезагрузите страницу.")
14             await websocket.close()
15             return
16
17         clients[websocket] = nickname
18         await broadcast(f"{nickname} присоединился к чату.")
19
20         try:
21             async for message in websocket:
22                 await broadcast(f"{nickname}: {message}")
23         except websockets.exceptions.ConnectionClosed:
24             pass
25         finally:
26             if websocket in clients:
27                 await broadcast(f"{nickname} покинул чат.")
28                 del clients[websocket]
29     except Exception as e:
30         print(f"Ошибка: {e}")
31
32     async def broadcast(message):
33         to_remove = []
34         for ws in clients:
35             try:
36                 await ws.send(message)
37             except:
38                 to_remove.append(ws)
39         for ws in to_remove:
40             if ws in clients:
41                 del clients[ws]
42
43     async def main():
44         async with websockets.serve(handler, "localhost", 8765):
45             print("Сервер запущен на ws://localhost:8765")
46             await asyncio.Future() # Бесконечное ожидание
47
48     if __name__ == "__main__":
49         asyncio.run(main())
```

Рисунок 9 – Код сервера в задании 4

Далее была создана html страница ее код представлен на рисунке 10.


```

PROD > Task_4 > <> index.html > html > link
1  <!DOCTYPE html>
2  <html lang="ru">
3  <link rel="stylesheet" href="styles.css">
4  <head>
5      <meta charset="UTF-8">
6      <title>Web-chat</title>
7      <style>
8          body {
9              font-family: Arial, sans-serif;
10             max-width: 800px;
11             margin: 0 auto;
12             padding: 20px;
13         }
14         #chat {
15             height: 400px;
16             border: 1px solid #ccc;
17             padding: 10px;
18             overflow-y: scroll;
19             margin-bottom: 10px;
20         }
21         input {
22             width: 70%;
23             padding: 8px;
24         }
25         button {
26             padding: 8px 15px;
27             background-color: #4CAF50;
28             color: white;
29             border: none;
30             cursor: pointer;
31         }
32     </style>
33 </head>
34 <body>
35     <div class="container">
36         <h2>Web Chat</h2>
37         <div id="chat"></div>
38         <div>
39             <input id="msg" type="text" placeholder="Введите сообщение...">
40             <button onclick="sendMessage()">Отправить</button>
41         </div>
42     </div>
43     <script src="chat.js"></script>
44 </body>
45 </html>

```

Рисунок 10 – Код html страницы

Так же для корректной работы был создан javascript файл, его код представлен на рисунке 11.

```

PROD > Task_4 > JS chatjs > ...
1  const ws = new WebSocket("ws://localhost:8765");
2  const chat = document.getElementById("chat");
3  const msgInput = document.getElementById("msg");
4
5  let nickname = ""; // Инициализируем переменную
6  let nicknameSet = false;
7
8  ws.onmessage = (event) => {
9      const msg = event.data;
10
11      if (!nicknameSet && msg === "Введите никнейм:") {
12          const inputNickname = prompt("Введите ваш никнейм:");
13          if (inputNickname) {
14              nickname = inputNickname; // Сохраняем введенный ник
15              ws.send(nickname);
16              nicknameSet = true;
17          }
18          return;
19      }
20
21      if (msg.includes("Никнейм уже занят")) {
22          alert(msg);
23          ws.close();
24          return;
25      }
26
27      const messageElement = document.createElement("div");
28      messageElement.textContent = msg;
29
30      // Определяем тип сообщения
31      if (msg.includes("присоединился") || msg.includes("покинул")) {
32          messageElement.className = "system-message";
33      } else if (nickname && msg.startsWith(`${nickname}:`)) {
34          messageElement.className = "user-message";
35      } else {
36          messageElement.className = "other-message";
37      }
38
39      chat.appendChild(messageElement);
40      chat.scrollTop = chat.scrollHeight;
41  };
42
43  function sendMessage() {
44      const text = msgInput.value.trim();
45      if (text && nicknameSet) {
46          ws.send(text);
47          msgInput.value = "";
48      }
49  }
50
51  msgInput.addEventListener("keydown", (e) => {
52      if (e.key === "Enter") sendMessage();
53  });
54
55  ws.onclose = () => {
56      const messageElement = document.createElement("div");
57      messageElement.className = "system-message";
58      messageElement.textContent = "Соединение закрыто";
59      chat.appendChild(messageElement);
60  };

```

Рисунок 11 – Javascript файл для подключения к серверу

Вывод

В ходе выполнения данной лабораторной работы были приобретены основы владения практическими навыками и умениями реализации webсерверов и использования сокетов.